北京工业大学耿丹学院

毕业设计（论文）

题目：基于大模型的校园智能问答系统的设计与实现

姓　　名　刘　奇

二级学院　信息工程学院

专　　业　计算机科学与技术

行政班级　计科 23-2 班

学　　号　230809215

指导教师　段雪丽

日　　期　2027 年 4 月

诚信承诺

本人 刘 奇 声明，本论文及其研究工作是由本人在导师指导下独立完成，论文所利用的一切资料均符合论文著作要求，且在参考文献中列出。

本人充分认识到学位论文造假、抄袭、代写等学术不端行为的严重后果，承诺对论文内容的真实性、原创性负责，愿意接受学校及教育主管部门的检查。

签名：刘奇

日期：2027 年 4 月

摘　要

随着智慧校园建设的推进，教务管理、校园生活、实验室使用和办事流程等信息大量分散在规章制度、通知公告与办事指南中，学生检索成本高，管理人员重复答疑多，不同渠道的回答口径也不一致。通用大语言模型具备较强的自然语言理解与生成能力，但缺乏校本知识，直接回答校园问题容易产生事实性错误，且难以说明答案来源。针对上述问题，本文设计并实现了一个基于大模型的校园智能问答系统。系统以学校公开资料为知识源，采用 Vue 3 与 FastAPI 构建前后端分离的 Web 应用，使用 MySQL 保存业务数据，使用向量数据库保存知识片段索引；在数据处理阶段完成文档解析、清洗、结构感知分块与向量化，在问答阶段采用 BM25 与稠密向量双路召回、倒数排序融合和交叉编码器重排，并将检索到的证据与问题一并送入大模型，通过提示词约束生成带引用编号的回答，证据不足时执行拒答。系统还包括用户与权限管理、资料管理、问答历史、满意度反馈和管理统计等功能。本文完成了系统的需求分析、总体设计、关键模块实现方案与测试方案设计，给出了检索、生成、引用溯源和权限控制的具体实现思路，并设计了纯大模型问答与不同检索策略的对照实验，为后续系统联调和效果验证提供了可复现的依据。

研究方向：教育信息化；检索增强生成

关键词：知识库；混合检索；重排序；引用溯源；受约束生成

Abstract

With the advancement of smart campus construction, information about academic affairs, campus life, laboratory use, and administrative procedures is scattered across regulations, notices, and service guides. Students spend considerable time searching for answers, and different channels sometimes provide inconsistent information. Generic large language models lack institution-specific knowledge; when asked campus-related questions, they may produce plausible but incorrect answers and usually cannot indicate the source. To address these problems, this thesis designs and implements a campus intelligent question answering system based on a large language model. The system uses publicly available campus documents as its knowledge source and adopts a front-end and back-end separated architecture built with Vue 3 and FastAPI. MySQL stores business data, and a vector database stores knowledge chunk indexes. Documents are parsed, cleaned, split in a structure-aware manner, and vectorized. For question answering, BM25 and dense vector retrieval are combined, reciprocal rank fusion merges the candidates, and a cross-encoder reranks the results. The question and retrieved evidence are sent to the large language model, and prompt constraints require the answer to cite the evidence; when evidence is insufficient, the system refuses to answer. The system also provides user and permission management, document management, question answering history, satisfaction feedback, and administrative statistics. This thesis completes requirement analysis, overall design, key module implementation scheme, and test plan design, and designs a comparative experiment between direct large-model answering and different retrieval strategies, providing a reproducible basis for subsequent integration and evaluation.

Research Area: Education Informatization; Retrieval-Augmented Generation

Key Words: Knowledge Base; Hybrid Retrieval; Reranking; Citation Tracing; Constrained Generation

目 录

摘 要 I
Abstract II
第 1 章 绪论 1
1.1 研究背景与意义 1
1.2 国内外研究现状 2
1.2.1 检索增强生成的研究进展 2
1.2.2 检索与重排序技术 2
1.2.3 垂直领域问答系统的应用 3
1.2.4 评价、安全与工程实现 4
1.2.5 现有研究的不足 4
1.3 研究内容与方法 5
1.4 论文组织结构 5
第 2 章 相关技术与理论基础 7
2.1 大语言模型与提示工程 7
2.2 检索增强生成 8
2.3 文本向量化与语义检索 9
2.4 混合检索与重排序 9
2.5 多轮对话与问题改写 11
2.6 Web 开发与权限控制 12
第 3 章 系统需求分析 13
3.1 可行性分析 13
3.1.1 技术可行性 13
3.1.2 经济可行性 13
3.1.3 操作可行性 14
3.1.4 进度可行性 14
3.2 用户角色与权限需求 14
3.3 功能需求分析 15
3.4 非功能需求分析 17
3.5 用例与业务流程 18
第 4 章 系统设计 20
4.1 总体架构设计 20
4.2 知识库与数据处理设计 21
4.3 检索增强问答流程设计 23
4.4 数据库与接口设计 24
4.5 权限与安全设计 27

第 5 章 系统实现 29
5.1 开发环境与项目结构 29
5.2 用户与权限管理 30
5.3 文档管理与知识库构建 31
5.4 混合检索与重排序实现 32
5.5 受约束生成、引用溯源与多轮会话 33
5.6 管理统计与系统配置 34
第 6 章 系统测试与效果评价 35
6.1 测试环境与测试方案 35
6.2 问答测试集与评价指标 37
6.3 对照实验设计 38
6.4 结果记录与分析方法 39
结 论 40
参考文献 42
致 谢 45

第 1 章　绪论

1.1 研究背景与意义

近年来，高校智慧校园建设持续推进，教务管理、校园生活、实验室使用和办事流程等事务逐步迁移到线上。然而，与学生日常使用密切相关的信息并没有因此变得集中：培养方案、学籍管理规定、考试安排、宿舍报修、校园网认证、实验室预约等内容，分别由教务、学工、后勤、图书馆和二级学院等不同部门发布，散落在规章制度、通知公告、办事指南和常见问答之中。文件数量多、条款分散、更新节奏不一致，学生往往只能逐页翻阅，或者向辅导员、班委和同学询问。这种依赖人工转述的信息获取方式不仅效率低，而且不同渠道给出的说法有时并不一致，容易造成误解。

通用大语言模型为校园信息获取提供了新的可能。以 Transformer 为基础的大模型能够理解自然语言问题，并以接近日常交流的方式组织答案，一定程度上降低了用户表达问题和理解答案的门槛。但通用模型的知识主要来自公开训练语料，并不掌握学校最新的校本文件；直接让它回答“补办学生证需要哪些材料”“某门课程重修如何申请”等问题，模型可能给出看似合理却没有依据的内容，即通常所说的“幻觉”。更关键的是，通用模型一般不会说明答案出自哪一份文件、哪一条规定，用户难以核实，学校也难以对回答质量进行追责。检索增强生成（Retrieval-Augmented Generation，RAG）的出现为这一问题提供了可行路径：系统先从外部知识库中检索与问题相关的证据，再把证据和问题一起交给大模型，让模型在给定材料的范围内组织回答[1]。在校园场景中引入 RAG，可以把“查文件”和“问模型”结合起来，使回答既有自然语言的易读性，又能追溯到具体资料[2]。

本课题的理论意义在于，把中文校园文档的分块、混合检索、重排序、受约束生成和引用溯源串成一条完整流程，并通过对照实验观察检索增强前后回答准确性和可核查性的变化，为 RAG 在校园垂直场景中的落地提供可复用的设计参考。实践意义则体现在三个方面：一是降低学生获取校园信息的成本，减少在

多个文件之间反复查找的时间；二是减轻辅导员和管理人员的重复答疑负担；三是通过问答历史、高频问题和满意度反馈，为学校改进制度宣传和服务流程提供数据依据。与训练或微调专用模型相比，基于成熟大模型接口和开源检索组件的方案实施成本较低，便于在本科毕业设计的周期内完成，也便于后续推广到其他高校。

1.2 国内外研究现状

1.2.1 检索增强生成的研究进展

检索增强生成的核心思想，是把参数化的大模型与非参数化的外部知识库结合起来。Lewis等提出的 RAG 框架由检索器和生成器两部分组成：检索器从维基百科的稠密向量索引中取出相关段落，生成器以问题与段落为条件产生答案[1]。这一工作说明，回答所需的证据不必全部固化在模型参数中，可以在推理阶段从外部资料现取。此后，围绕 RAG 的优化逐渐形成一条较完整的技术路线。袁乐等将 RAG 的流程拆分为分块、向量化、建索引、查询处理、块选择和提示构建等环节，并从预检索增强、检索器增强、检索策略增强、索引增强、检索后增强和大模型增强六个方面归纳了优化方法[3]。张犬俊等则从软件工程的角度梳理了 RAG 在需求分析、代码生成、测试和维护等任务中的应用，指出检索质量与上下文组织方式对最终效果具有决定性影响[4]。

在生成控制方面，Asai 等提出的 Self-RAG 让模型按需发起检索，并通过反思标记评价检索段落与自身生成内容的相关性和事实性[5]。该方法在多个知识密集型任务上取得了优于常规 RAG 的表现，但需要额外的训练和多次模型调用，成本较高。对于本科课题而言，更现实的做法是借鉴其“先判断证据是否充分”的思想，通过提示词约束、证据阈值和拒答策略实现轻量化的可控生成。总体来看，RAG 已经形成较为清晰的技术框架，但面向校园这类资料格式多样、更新频繁、权限要求明确的场景，仍需要针对性的工程化设计。

1.2.2 检索与重排序技术

检索环节决定了送入生成模型的证据质量。早期的问答系统多采用关键词匹配或 BM25 等稀疏检索方法，这类方法对专有名词、文件编号和条款序号比较敏感，但难以处理同义表达和语义泛化。Karpukhin 等提出的密集段落检索（Dense Passage Retrieval，DPR）使用双塔结构，将问题和段落分别编码为稠密向量，再

通过向量相似度召回候选段落[6]。与 BM25 相比，DPR 不再依赖词语是否重合，能够在一定程度上理解语义相近但措辞不同的问法。中文场景下的向量化则可以直接借助已有资源。Xiao 等构建的 C-Pack 包含中文评测基准、大规模训练语料和 BGE 系列预训练模型，为中文语义表示提供了成套工具[7]。本课题选用 BGE 系列模型完成知识片段和问题的向量化，避免从零训练嵌入模型。

单一检索通道各有短板。郜洪奎等指出，单纯依赖向量表示难以覆盖检索任务的复杂性，而且缺少有效的排序机制，容易把无关信息带入上下文；他们先用多路径召回扩大候选范围，再用双层网络对候选进行多轮排序[8]。张健等发现，按段落切分比按固定长度切分更有利于保持语义完整，检索知识放在问题之前时采用逆序重排序可以改善回答准确率[9]。孙飒等面向企业知识库提出结构感知的自适应分块、由摘要与细粒度内容构成的层次化父子块索引，以及双通道混合检索和交叉重排方案，以解决固定窗口切分把完整操作步骤拆散、扁平化索引丢失页面层级的问题[10]。校园文件同样具有标题层级和条款结构，这些工作为本课题的分块策略和混合检索设计提供了直接参考。

1.2.3 垂直领域问答系统的应用

在校园和垂直领域，RAG已经被用于多项知识密集型问答任务。白晶等把高校知识问答系统自下而上划分为数据层、检索层、模型层和应用层，并建立了覆盖性能指标、用户体验和知识质量的三维评价体系[2]。宋保仁和冷静以四川高校图书馆特色数据库为例，分析了 RAG 在私有知识库建设中的应用与边界，认为该技术提升了特色资源的检索效率和知识组织水平，同时提醒要配套可审查的生成路径和协同治理机制[11]。这一结论对本课题具有警示意义：校园资料的范围、更新责任和答案可核查性，必须在系统设计阶段就明确下来。

秦晓瑞等面向结构复杂、语义严谨的航空标准文档，设计了结构路径感知的知识库，用章节结构和标题链条支持语境追溯，并在检索阶段融合关键词与语义信号，由大模型判断是否需要拆解子问题、发起多轮检索[12]。甘克勤等把对话历史管理单独做成模块，通过主题建模和序列编码分析会话的对象、主题和结构，使系统在多轮问答中能够理解指代和上下文[13]。罗云等提出的 SV-RAG 在切分时保留层次结构元数据，检索时采用双轨混合检索，生成时要求模型先引用核心原文再展开说明，以增强回答的忠实性[14]。这种“先引用、后阐述”的顺序，对本课题的引用溯源设计很有启发。

此外，董永涛等设计了舰艇装备故障智能问答系统，彭琳讨论了企业级知识

库的构建与优化，陶金涛等将 RAG 用于新疆地质矿产问答，王祥昊等将其用于铁路应急处置策略生成，窦嘉铭等则把知识图谱与 RAG 结合，用于配网检修辅助决策[15, 16, 17, 18, 19]。这些工作说明，RAG 在专业资料密集、答案准确性要求高的场景中具有较好的适用性；但校园场景的特殊性在于资料以中文行政文本为主、更新频繁、用户角色多样，现有研究对这类资料的持续建库和权限隔离讨论得还不够充分。

1.2.4 评价、安全与工程实现

如何评价 RAG 系统的效果，是另一条重要线索。Es 等提出的 RAGAS 不依赖人工标注的参考答案，围绕忠实性、答案相关性和上下文相关性三个维度自动打分，分别对应答案能否由上下文推出、是否回答了问题、检索内容是否集中于问题[20]。这套指标为本课题的效果测试提供了参考。不过，校园问答还需要关注引用是否真实、证据不足时能否拒答，因此仅靠通用指标并不够，需要结合人工标注的校园问题测试集进行补充评价。

在系统安全和工程实现方面，卢万有给出了基于 JWT 的无状态认证与 RBAC 角色权限控制方案，身份信息放在令牌中在前后端之间传递，后端通过拦截器完成校验[21]。廖明焱针对企业 RAG 的权限控制缺失和敏感信息泄露问题，设计了多层级权限控制的安全检索架构与智能脱敏方法[22]。魏巍则从云资源管理的角度讨论了 RBAC 模型的权限设计[23]。这些工作说明，知识库系统的权限控制不能只做在页面入口，还要落实到检索和证据返回环节。张宁以 LangChain 为框架，接入本地部署的 DeepSeek 模型，用 Chroma 存放知识向量，前后端采用 Vue 3 和 FastAPI，其技术选型与本课题较为接近[24]。吴海周等把系统分为索引、检索、生成三个模块分别优化，采用语义向量构建、动态 Top-k 检索和上下文融合生成等方法，说明检索条数、上下文长度等参数需要结合具体数据调整[25]。LangChain 官方文档则为文档加载、切分、向量存储和检索链的组织提供了工程参考[26]。

1.2.5 现有研究的不足

综合上述工作可以看出，RAG 在知识密集型问答中的有效性已经得到较多验证，但放到校园场景中仍有三个问题没有很好解决。第一，校园资料来自不同部门，格式不统一、更新频繁，现有研究对这类资料如何持续建库、如何保留标题层级和条款结构讲得不够细。第二，不少系统更关注回答读起来是否顺畅，对

引用是否真实、证据不足时能否拒答、不同角色的资料是否需要隔离考虑得较少。第三，效果评价大多沿用通用指标，缺少针对本校常见问题和引用正确性的测试集。本课题围绕这三点展开：以学校公开资料建立可控的校园知识库，用关键词检索和向量检索双路召回后再重排，生成时限定只能使用检索到的证据并标出引用编号，最后从检索命中、答案正确、引用正确、拒答准确和响应时延等方面设计验证方案。

1.3 研究内容与方法

本课题的研究内容可以概括为四个方面。第一，校园知识库构建。研究 PDF、Word 和纯文本等常见格式的解析方法，设计保留标题层级、页码、分类和更新时间的结构感知分块策略，并完成文本向量化和索引构建。第二，混合检索与重排序。在 BM25 稀疏检索和 BGE 稠密向量检索两条通道上分别召回候选，使用倒数排序融合（Reciprocal Rank Fusion，RRF）合并结果，再通过交叉编码器对顶部候选进行重排。第三，带引用的受约束生成。设计包含问题、证据和输出约束的提示词模板，要求模型只依据证据回答、为关键结论标注引用编号，并在证据不足时拒答；同时处理多轮对话中的指代和追问。第四，系统实现与效果评价。采用 Vue 3、FastAPI、MySQL 和向量数据库完成 Web 系统，设计功能测试、性能测试和对照实验，比较纯大模型问答与不同检索策略的差异。

研究方法上，本文采用文献分析、需求调研、原型开发和对照实验相结合的方式。文献分析用于梳理 RAG、稠密检索、混合检索、重排序和评价指标的研究基础；需求调研结合任务书、开题报告和校园常见问题，明确学生与管理员的使用场景；原型开发按照软件工程的流程完成需求分析、总体设计、详细设计、编码实现和联调；对照实验则在同一测试集上比较不同方案，尽量保证变量可控、结果可复现。

1.4 论文组织结构

本文共分为六章，另设结论、参考文献和致谢。第 1 章介绍研究背景、国内外研究现状、研究内容与方法，说明课题的来源和意义。第 2 章介绍大语言模型、检索增强生成、文本向量化、混合检索与重排序、多轮对话和 Web 权限控制等关键技术，为后续设计提供理论依据。第 3 章从可行性、用户角色、功能需求、非功能需求和业务流程等方面完成系统需求分析。第 4 章给出系统总体架构、知

识库与数据处理、检索增强问答流程、数据库与接口、权限与安全等设计。第 5 章说明开发环境、项目结构以及用户权限、文档管理、混合检索、受约束生成和管理统计等模块的实现。第 6 章给出测试环境、功能与性能测试、问答测试集与评价指标、对照实验和结果分析方法。结论部分总结全文工作，指出不足并展望后续改进方向。

第 2 章　相关技术与理论基础

本章介绍系统涉及的关键技术。首先说明大语言模型的基本原理和提示工程方法，然后分析检索增强生成的流程与适用边界，接着讨论文本向量化、混合检索、重排序和多轮对话中的关键问题，最后介绍前后端分离架构下的身份认证与权限控制。这些内容构成后续需求分析和系统设计的理论基础。

2.1 大语言模型与提示工程

大语言模型通常以 Transformer 的解码器结构为基础，通过在海量文本上预测下一个词元进行预训练。模型在生成时根据已有上下文逐个产生词元，并利用注意力机制捕捉长距离依赖。经过指令微调和对齐训练后，模型能够按照用户的自然语言指令完成问答、摘要、改写和代码生成等任务。对于问答系统而言，大模型的价值主要在于语言理解和组织能力：它可以把用户口语化的问题转换为规范的表述，也可以把检索到的多条证据整合成连贯的回答。

大模型的输出具有概率性，受到温度、Top-p、最大生成长度等参数影响。温度越低，输出越稳定、保守；温度越高，输出越发散，但事实性风险也随之增加。校园问答要求答案准确、口径统一，因此需要把温度调低，并通过系统提示明确角色、任务、证据范围和输出格式。提示工程在这里的作用不是“让模型更聪明”，而是把任务边界说清楚：哪些内容必须依据证据，哪些内容不能推测，证据不足时应该如何回复，引用编号应该放在什么位置。对于结构化输出，还可以要求模型按照固定的 JSON 字段返回答案、引用编号和置信度，便于后端解析和校验。

本课题不训练或微调大模型，原因有三点。其一，训练大模型需要大规模语料和算力，超出一名本科生毕业设计的条件；其二，校园资料规模相对有限，通过检索增强即可把校本知识注入回答过程，没有必要把知识固化到参数中；其三，使用成熟的大模型接口可以让研究重点集中在知识库构建、检索融合、引用溯源和评价方法上，符合任务书“不要求训练或微调大模型”的要求。为避免对单一模型服务产生依赖，系统在模型接口层做抽象，配置项中只保存服务地址、

模型名称和密钥引用，更换模型时不需要修改业务代码。

2.2 检索增强生成

检索增强生成把生成过程拆成“索引—检索—生成”三个阶段。索引阶段把外部文档切分成片段，计算向量或建立倒排索引，写入向量数据库或搜索引擎；检索阶段根据用户问题召回相关片段，并可选择性地进行融合和重排；生成阶段把问题、候选证据和输出约束组织成提示词，交给大模型生成答案。与纯参数化模型相比，RAG的优势在于知识可以随时更新、答案可以附带出处、错误可以定位到具体资料。其基本流程如图 2-1 所示。

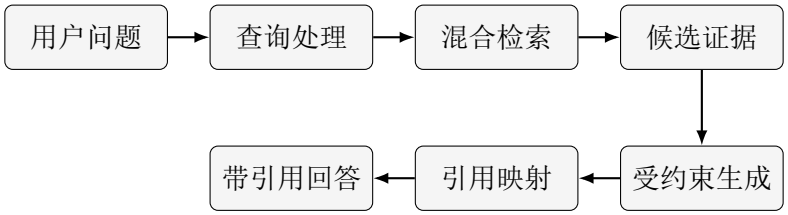


图 2-1 RAG 问答的基本流程

RAG 并不能自动消除幻觉。检索不到相关证据、召回内容与问题无关、证据之间互相矛盾、模型没有遵守提示词约束，都会导致回答失败。因此，一个可用的 RAG 系统需要在每个环节设置可控点：检索阶段要有召回率和排序质量指标，生成阶段要有证据约束和拒答策略，输出阶段要有引用校验。Self-RAG 通过反思标记让模型判断是否需要检索、证据是否相关、生成内容是否有支持[5]，其思想可以借鉴，但其训练成本较高。本课题采用轻量化的替代方案：在检索后设置相关性阈值，在提示词中要求“只依据证据回答”，在输出后检查引用编号是否落在候选证据范围内，任一环节不满足条件时转为拒答或提示用户补充信息。

从知识组织方式看，RAG 也可以与知识图谱结合。知识图谱擅长表达实体之间的结构化关系，适合处理“某学院有哪些专业”“某课程的先修课程是什么”这类关系型问题；RAG 擅长处理非结构化文本中的解释性内容[19]。校园问答以规章制度和办事指南为主，答案往往是一段说明性文字，因此本课题以文本片段检索为主，暂不引入知识图谱，以控制系统复杂度。

2.3 文本向量化与语义检索

文本向量化把自然语言映射到固定维度的实数向量，使语义相近的文本在向量空间中距离较近。DPR 采用双塔结构，问题和段落分别经过独立的编码器，训练目标是让问题与正确段落的向量相似度高于错误段落[6]。在推理阶段，所有段落向量可以预先计算并建立索引，用户提问时只需编码问题，再在向量库中做近邻搜索。与 BM25 相比，稠密检索不要求问题与段落共享相同的词语，因此对同义改写、口语化表达和跨句语义匹配更友好。

中文向量化模型的选择需要考虑语料领域和部署条件。C-Pack 提供了面向中文的评测基准、训练语料和 BGE 系列模型，覆盖语义相似度、检索、聚类等多种任务[7]。本课题优先选用参数量适中的 BGE 中文模型，在本地 CPU 上完成知识片段和问题的编码，以降低对外部服务的依赖。对于长度超过模型输入上限的文档，需要先分块再编码；分块过粗会引入无关内容，分块过细则会破坏上下文，因此分块策略需要结合文档结构和使用场景确定。

在向量检索中，常用余弦相似度衡量问题向量与片段向量的接近程度。设问题向量为 $\mathbf{q}$，片段向量为 $\mathbf{d}$，其余弦相似度定义为

$$\cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{\|\mathbf{q}\|\,\|\mathbf{d}\|}. \tag{2-1}$$

当向量已经归一化时，余弦相似度等价于内积，计算量更小。实际系统中通常取相似度最高的 k 个片段作为候选，k 的取值需要在召回率和上下文长度之间权衡。k 过小可能漏掉关键证据，k 过大则会把噪声带入生成阶段，增加模型混淆和响应时延。

向量数据库负责保存片段向量、原文和元数据，并支持按相似度检索。常见的实现包括 Chroma、Milvus、FAISS 等。Chroma 部署简单、提供 Python 接口，适合本科课题的原型开发；Milvus 更适合大规模数据和高并发场景。本课题选用 Chroma 作为默认实现，同时在数据访问层保留接口抽象，便于后续替换。除向量外，每个片段还需要保存文档编号、标题路径、页码、分类、更新时间和权限标签等元数据，这些信息既用于结果展示，也用于检索过滤和引用溯源。

2.4 混合检索与重排序

稀疏检索和稠密检索的互补性，是混合检索的依据。BM25 基于词频和逆文档频率计算相关性，对专有名词、文件编号、条款序号等精确匹配非常有效，但

无法理解同义表达；稠密检索能够捕捉语义相似性，却可能忽略关键的数字、名称和编号。把两条通道的候选合并，可以在不显著增加成本的前提下提高召回率。BM25 的常用形式为

$$\mathrm{BM25}(q,d)=\sum_{t\in q}\mathrm{IDF}(t)\cdot\frac{f(t,d)\,(k_1+1)}{f(t,d)+k_1\left(1-b+b\cdot\frac{|d|}{\mathrm{avgdl}}\right)},\tag{2-2}$$

其中 $f(t,d)$ 表示词项 t 在文档 d 中的词频，$|d|$ 为文档长度，avgdl 为平均文档长度，k_1 和 b 为调节参数。BM25 不需要训练，计算速度快，适合作为混合检索中的精确匹配通道。

两条通道的候选不能直接按分数相加，因为 BM25 分数和余弦相似度的量纲不同。常用的融合方法是倒数排序融合，它只利用候选在各通道中的排名，不依赖原始分数：

$$\mathrm{RRF}(d)=\sum_{r\in R}\frac{1}{k+\mathrm{rank}_r(d)},\tag{2-3}$$

其中 R 为召回通道集合，$\mathrm{rank}_r(d)$ 为文档 d 在通道 r 中的排名，k 为平滑常数，通常取 60。RRF 的优点是实现简单、对分数尺度不敏感，适合异构检索通道的融合。融合后的候选还需要经过一次重排序。交叉编码器把问题和候选片段拼接后一起输入模型，能够建模两者之间的细粒度交互，排序精度通常高于双塔模型，但计算成本也更高，因此只对顶部若干条候选使用。

图 2-2 对比了双塔检索与交叉重排的差异。双塔结构可以预先编码文档，适合大规模召回；交叉编码器必须在线计算问题与每个候选的匹配分数，适合对少量候选做精排。本课题采用“双路召回—RRF 融合—交叉编码器重排—Top-k 截断”的链路，在召回率和时延之间取得平衡。

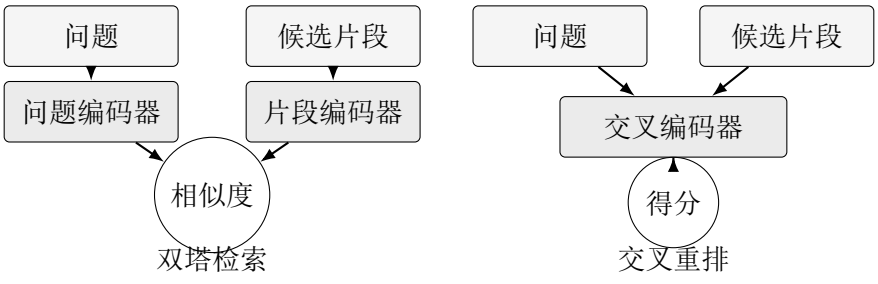


图 2-2 双塔检索与交叉重排的结构对比

表 2-1 从匹配方式、优点、局限和适用场景四个方面比较了三种检索方案。可以看出，混合检索并不是简单地“二选一”，而是把稀疏通道的精确性和稠密通道的语义泛化能力结合起来，再通过重排序提升顶部结果的相关性。

表 2-1 三种检索方案的对比

对比维度	BM25 稀疏检索	稠密向量检索	混合检索
匹配方式	词项匹配、逆文档频率加权	语义向量相似度	双路召回后融合
主要优点	对专有名词、编号、条款敏感；无需训练	能处理同义表达和口语化提问	兼顾精确匹配与语义泛化
主要局限	难以处理同义改写；受分词影响	对数字、编号不敏感；依赖嵌入模型	参数增多，融合与重排带来额外时延
适用场景	文件编号、条款查询	概念解释、流程咨询	校园综合问答

2.5 多轮对话与问题改写

校园咨询往往不是一问一答。用户可能先问“国家助学金的申请条件是什么”，接着追问“那需要哪些材料”，再问“什么时候截止”。如果系统只把当前问题送入检索，第二、第三个问题缺少主语和限定条件，检索结果很容易偏离主题。多轮对话管理的任务，就是把当前问题与最近的会话历史结合起来，改写成可以独立检索的完整问题，同时保留原始问题，以便追溯用户真实意图[13]。

常见做法是保留最近若干轮对话，把用户当前问题、上一轮问题和系统回答一起送入改写模型，要求模型补全指代、省略和隐含条件，输出一个不依赖上下文的自包含问题。改写后的查询用于检索，原始问题用于最终回答和日志记录。这样既能提高召回率，又不会因为改写错误而丢失原始语义。对于“那研究生呢”这类对比式追问，改写结果应保留前文的主题，只替换限定对象；对于“这个流程要多久”这类指代式追问，改写结果应把“这个流程”替换为上一轮的具体事项。

多轮对话还涉及历史记录的管理。系统需要保存会话、消息、引用和反馈，支持用户查看历史、继续提问和重新发起会话。为了避免上下文过长，通常只保留最近 N 轮对话，并对早期内容做摘要。摘要可以压缩 token 消耗，但也可能丢失细节，因此本课题仅在会话轮次较多时启用摘要，并在引用展示中保留原始证据，保证用户仍能核对答案来源。

2.6 Web 开发与权限控制

系统采用前后端分离的 B/S 架构。前端使用 Vue 3 和 Element Plus 构建页面，负责登录、问答、引用展开、历史记录和管理后台等交互；后端使用 Python 与 FastAPI 提供 RESTful 接口，负责身份认证、业务逻辑、文档处理、检索和生成；MySQL 保存用户、角色、文档、会话、消息、反馈和配置等关系数据；向量数据库保存知识片段索引；大模型、嵌入模型和重排序模型通过统一的模型服务层访问。前后端通过 JSON 交换数据，接口使用统一的响应结构和错误码，便于前端处理异常。

身份认证采用 JWT。用户登录成功后，后端校验密码哈希，签发包含用户编号、角色和过期时间的令牌；前端在后续请求的 Authorization 头中携带令牌；后端通过依赖注入解析令牌、校验签名和有效期，并把当前用户信息注入请求上下文[21]。JWT 是无状态的，服务端不需要保存会话，适合前后端分离和水平扩展；但令牌一旦签发，在过期前无法主动失效，因此需要设置较短的有效期，并配合刷新令牌或黑名单机制处理注销和权限变更。

授权采用基于角色的访问控制（Role-Based Access Control，RBAC）。系统定义学生、教师和管理员等角色，为角色分配接口权限，为用户分配角色。后端在处理请求前检查当前角色是否具备所需权限，前端根据权限隐藏不可操作的菜单和按钮。权限控制不能只做在页面层，否则用户可以直接调用接口绕过限制；因此文档上传、删除、统计查看等敏感操作必须在后端再次校验[23]。对于知识库检索，还需要在检索阶段按文档的权限标签过滤，避免低权限用户通过问答间接获取受限内容[22]。

除认证授权外，系统还需要处理以下安全问题：密码使用 bcrypt 或 Argon2 等算法加盐哈希存储，不保存明文；大模型和向量库的密钥通过环境变量或配置文件注入，不写入代码仓库；上传文件校验扩展名、MIME 类型和大小，解析时限制页数和超时时间，防止恶意文件导致服务不可用；接口对输入长度、分页参数和检索条数做范围校验，防止注入和资源耗尽；日志对手机号、身份证号等敏感字段脱敏，避免二次泄露。这些措施将在第 4 章的安全设计中进一步展开。

第 3 章　系统需求分析

需求分析是连接课题目标与系统设计的中间环节。本章从可行性、用户角色、功能需求、非功能需求和业务流程五个方面展开，明确系统“为谁做、做什么、做到什么程度”，为第 4 章的设计提供依据。需求来源包括任务书、开题报告、校园常见问题以及对学生和管理人员的访谈记录。

3.1 可行性分析

3.1.1 技术可行性

系统所需的技术组件均有成熟方案。前端采用 Vue 3 与 Element Plus，能够快速构建表单、列表、对话框和聊天界面；后端采用 Python 与 FastAPI，异步框架适合处理文档解析、向量检索和大模型调用等 I/O 密集型任务；MySQL 适合保存结构化业务数据；Chroma 提供轻量的向量存储和相似度检索接口；BGE 中文嵌入模型和交叉编码器可以在 CPU 上运行，也可以按需替换为其他模型；大模型通过 HTTP 接口调用，不需要本地显卡。上述组件均为开源或公开可用的服务，文档齐全，社区活跃，出现问题时容易找到解决方案。

从开发能力看，本人在本科阶段学习了软件工程、数据库、Web 开发、数据结构与算法等课程，能够完成前后端分离项目的设计与开发；了解 Python、JavaScript、SQL 和 RESTful 接口的基本用法；通过文献阅读和实验，掌握了文档解析、文本分块、向量化、检索和提示词构造的基本流程。课题不要求训练或微调大模型，技术风险主要集中在文档解析质量和检索效果调优上，这两部分都可以通过分阶段迭代解决。

3.1.2 经济可行性

系统开发和运行的成本较低。开发阶段使用个人计算机和开源软件，不需要购买服务器和商业数据库；嵌入模型和重排序模型可以在本地 CPU 上运行，不需要 GPU；大模型按调用量计费，测试阶段的使用量有限。部署阶段可以运行在学校已有的虚拟机上，也可以使用低成本的云主机；向量数据库和 MySQL 均

可单机部署，硬件要求不高。与采购商业问答系统相比，本方案的一次性投入和长期维护成本都更容易控制。

3.1.3 操作可行性

系统采用 B/S 架构，用户通过浏览器即可访问，不需要安装客户端。学生端界面以对话为主，输入问题后直接查看答案和引用来源，操作路径短；管理端采用常见的后台布局，资料上传、分类、知识库构建和统计查看都以表单和列表呈现，符合管理人员的使用习惯。系统提供明确的错误提示和空状态引导，例如检索无结果时提示用户换一种问法，文档解析失败时显示失败原因和重试按钮，降低使用门槛。

3.1.4 进度可行性

按照任务书的进度安排，2026 年 9 月至 10 月完成调研、开题和需求分析，2026 年 10 月至 11 月完成核心模块开发，2026 年 12 月至 2027 年 2 月完成测试、论文和修改，2027 年 3 月完成查重、答辩和归档。本课题的功能范围明确，技术路线成熟，关键模块可以并行开发：文档处理和知识库构建、检索与重排序、问答与会话、管理统计相对独立，便于分阶段联调。只要控制好知识库范围和测试集规模，进度风险可控。

3.2 用户角色与权限需求

系统面向三类用户。学生是主要使用者，可以登录、提问、查看引用、继续追问、查询历史和提交满意度反馈，不能上传或删除知识资料。教师可以查看与自己课程或业务相关的问答统计，协助维护常见问题，但不具备用户管理和系统配置权限。管理员负责资料管理、知识库构建、用户与角色管理、统计分析和参数配置，是权限最高的角色。为避免权限过大，系统可以设置一名超级管理员，只用于初始化和应急处理，日常管理由普通管理员完成。

角色与功能的对应关系如表 3-1 所示。“查看”表示只能读取，“维护”表示可以新增、修改和删除，“管理”表示可以分配权限或修改系统级配置。权限控制采用“角色—权限—接口”三级映射：角色绑定权限集合，权限对应后端接口，接口在处理请求前校验当前用户是否具备相应权限。前端根据权限隐藏菜单和按钮，但后端校验才是最终防线。

表 3-1 角色与功能权限矩阵

功能模块	学生	教师	管理员	超级管理员
登录与个人信息	使用	使用	使用	使用
智能问答与追问	使用	使用	使用	使用
引用来源查看	查看	查看	查看	查看
问答历史与反馈	维护	维护	查看	查看
资料上传与分类	无	无	维护	维护
知识库构建与重建	无	无	维护	维护
高频问题与统计	无	查看	查看	查看
用户与角色管理	无	无	管理	管理
系统参数与密钥配置	无	无	维护	管理

除功能权限外，知识资料还需要内容级权限。校园公开制度、办事指南可以对所有登录用户开放；涉及教师评价、学生资助名单等敏感内容的文档，应限制在特定角色范围内。检索时先按用户角色过滤候选片段，再计算相似度，避免低权限用户通过问答间接获取受限信息。文档的权限标签由管理员在 upload 时设置，默认继承所属分类的权限，特殊文档可以单独覆盖。

3.3 功能需求分析

根据任务书和开题报告，系统需要实现用户管理、资料管理、知识库构建、智能问答、引用溯源、会话反馈和统计分析等功能。下面按“输入—处理—输出—异常”的方式逐项说明，功能编号与后续设计和测试用例保持一致。

（1）用户登录与权限管理（FR-01）。输入为账号、密码和验证码；处理时校验账号状态、密码哈希和验证码，签发 JWT，并根据角色加载权限；输出为令牌、用户信息和菜单权限；异常包括账号不存在、密码错误、账号被禁用、令牌过期，系统应返回明确的错误码并记录登录日志。

（2）校园资料上传与分类（FR-02）。输入为 PDF、Word 或纯文本文件，以及标题、分类、权限标签和备注；处理时校验文件类型和大小，保存原文件，创建文档记录，触发解析任务；输出为文档编号和解析状态；异常包括格式不支持、文件损坏、大小超限、重复上传，系统应拒绝或提示，并允许用户重试。

（3）文档解析、分块与向量化（FR-03）。输入为待处理的文档；处理时提取文本，清洗页眉页脚和多余空白，按标题层级和段落结构分块，保留页码和标题路径，调用嵌入模型生成向量，写入向量数据库；输出为知识片段和索引；异常包括解析失败、编码错误、模型服务不可用，系统应记录失败原因，支持断点重试和幂等重建。

（4）自然语言问答（FR-04）。输入为用户问题、会话编号和可选的分类过滤条件；处理时进行问题改写、双路召回、RRF 融合、交叉编码器重排，组装提示词并调用大模型；输出为答案、引用编号、候选证据和耗时；异常包括检索为空、模型超时、模型返回格式错误，系统应降级为拒答或提示稍后重试，不能返回无依据的内容。

（5）引用溯源与来源展示（FR-05）。输入为生成结果中的引用编号；处理时把编号映射回知识片段，校验片段是否属于本次候选集合，截取原文摘要；输出为可展开的来源卡片，包含文档标题、章节路径、页码和原文片段；异常包括编号越界、片段被删除、权限变化，系统应隐藏无效引用并提示用户。

（6）问答历史、继续提问与满意度反馈（FR-06）。输入为用户编号、会话编号和反馈值；处理时保存会话、消息、引用和反馈，支持按时间、关键词和分类查询；输出为历史列表、会话详情和反馈统计；异常包括会话不存在、无权查看、重复提交，系统应做幂等处理并返回统一错误信息。

（7）管理统计与系统配置（FR-07）。输入为时间范围、分类和统计维度；处理时聚合高频问题、未解决问题、知识库使用量、满意度和平均响应时延；输出为图表和明细列表；配置项包括检索条数、融合常数、重排阈值、模型名称、超时时间和开关；异常包括时间范围非法、配置值越界，系统应拒绝保存并给出取值范围。

上述功能的优先级和验收标准如表 3-2 所示。优先级“高”表示首版必须完成，“中”表示在核心链路稳定后完成，“低”表示可在后续版本迭代。

表 3-2 功能需求清单

编号	功能名称	主要验收标准	优先级
FR-01	登录与权限管理	正确签发和校验令牌；越权接口返回 403；登录失败有提示	高
FR-02	资料上传与分类	支持 PDF、DOCX、TXT；分类和权限标签可编辑；重复文件有提示	高
FR-03	解析、分块与向量化	文本提取完整；分块保留标题和页码；失败可重试	高
FR-04	自然语言问答	8 秒内返回；回答带引用；证据不足时拒答	高
FR-05	引用溯源与展示	引用可定位到文档、章节和页码；无效引用不展示	高
FR-06	历史、追问与反馈	历史可查询；多轮追问能理解指代；反馈可统计	中
FR-07	统计与系统配置	高频问题、未解决问题、满意度可查看；配置有范围校验	中

3.4 非功能需求分析

非功能需求决定系统是否“好用”和“可维护”。结合校园问答的使用场景，本文提出以下要求。

（1）性能。单次问答的平均响应时间不超过 8 秒，其中检索与重排不超过 2 秒，大模型生成不超过 6 秒；文档解析和向量化采用异步任务，不阻塞用户操作；管理端统计查询在 3 秒内返回。对于超过 8 秒的请求，前端显示加载状态和取消按钮，后端记录超时原因。

（2）并发与可用性。系统面向校内用户，按 50 个并发问答请求设计；采用无状态后端和连接池，支持水平扩展；关键服务出现故障时，问答接口降级为“仅返回检索片段”，保证用户仍能查看资料；系统可用性目标为 99%。

（3）可靠性。文档处理、向量化和模型调用都可能失败，系统需要记录任务状态、失败原因和重试次数，支持手动重试和定时补偿；问答记录、反馈和引用在事务中保存，不能因为一次模型调用失败而丢失；向量库和 MySQL 的数据需要定期备份。

（4）安全性。所有接口除登录外都需要认证；密码加盐哈希存储；大模型密钥和数据库密码通过环境变量注入；上传文件做类型、大小和内容校验；检索结果按权限过滤；日志脱敏；对登录、上传、问答等接口做频率限制，防止暴力破解和资源滥用。

（5）可维护性。前后端分离，接口使用统一的响应结构和错误码；业务逻辑按模块划分，模型服务、向量库和文档解析器都通过接口抽象，便于替换；关键操作记录结构化日志，日志包含请求编号、用户编号、耗时和错误码，方便排查。

（6）可扩展性。知识库支持新增分类和文档类型；检索参数、模型名称和提示词模板可以在管理端配置；向量库、嵌入模型和大模型服务可以替换；后续可以接入知识图谱、语音输入和多模态文档，而不需要重写问答主流程。

（7）易用性。界面以对话为核心，问题输入框支持多行和快捷发送；答案中的引用编号可点击展开；空状态、加载状态和错误状态都有明确提示；管理端表格支持筛选、分页和批量操作。

3.5 用例与业务流程

系统的用例关系如图 3-1 所示。学生的主要用例包括登录、提问、查看引用、继续追问、查询历史和提交反馈；教师可以查看统计和协助维护常见问题；管理员在此基础上增加资料管理、知识库构建、用户角色管理和系统配置。所有用户都需要先登录，问答和引用查看依赖于知识库中存在可检索的资料。

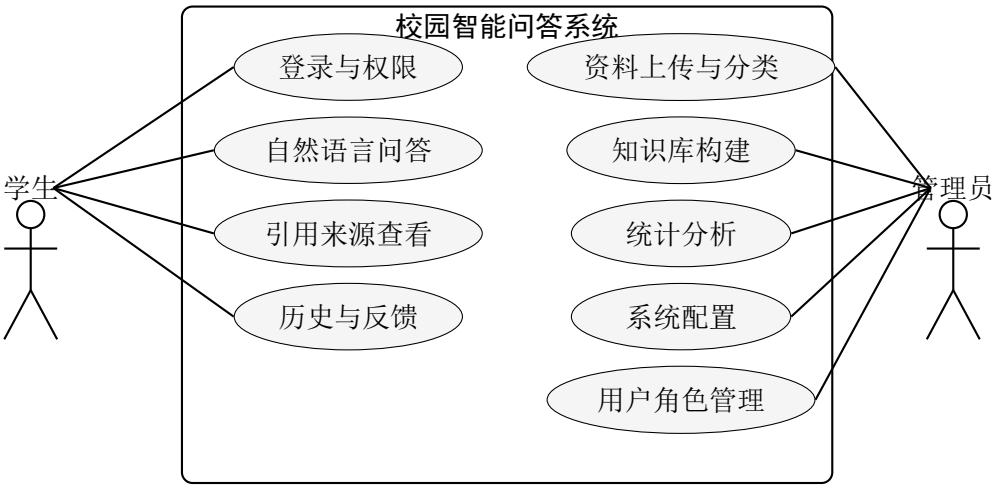


图 3-1 系统用例图

问答业务流程如图 3-2 所示。用户提交问题后，系统先结合最近会话做问题

改写，再分别执行 BM25 召回和向量召回，用 RRF 合并候选并重排。若最高相关性低于阈值或候选为空，系统不调用大模型，直接返回“暂时找不到依据”的提示，并给出可能的分类建议。若证据充足，系统把问题、证据和输出约束组装成提示词，调用大模型生成答案。生成后再做引用校验：引用编号必须落在候选证据范围内，答案中的关键结论必须能对应到至少一条证据。校验通过则返回答案和来源卡片；校验不通过则降级为摘录式回答，或提示用户补充信息。

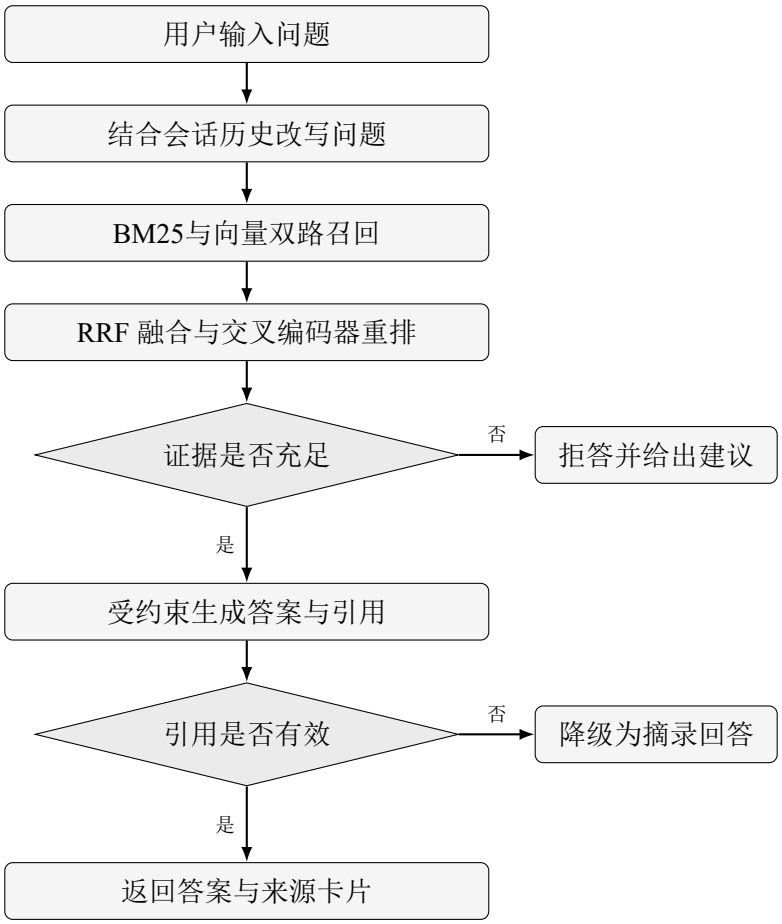


图 3-2 问答业务流程图

除主成功场景外，还需要考虑三类异常。第一，检索为空。可能原因是知识库没有相关资料，或者问题表述与资料差异过大。系统应提示用户换一种问法、选择分类，或建议联系对应部门，不能强行让模型回答。第二，模型超时或返回格式错误。系统应重试一次，仍失败则返回检索到的原始片段，并记录失败日志。第三，引用校验失败。可能是模型编造了编号，也可能是证据在生成过程中被删除。系统应隐藏无效引用，降级为摘录式回答，并把这次会话标记为“需人工检查”，供管理员分析。

第 4 章　系统设计

本章在第 3 章需求分析的基础上，给出系统的总体设计和详细设计。首先说明分层架构和模块划分，然后依次设计知识库与数据处理流程、检索增强问答流程、数据库与接口、权限与安全，最后给出关键参数和异常处理策略。设计遵循“高内聚、低耦合、可替换、可追溯”的原则，尽量把与具体模型服务相关的部分隔离在接口层之后。

4.1 总体架构设计

系统采用前后端分离的分层架构，自上而下分为表现层、接口层、业务层、数据与模型层，另设贯穿各层的安全与运维支撑。总体架构如图 4-1 所示。

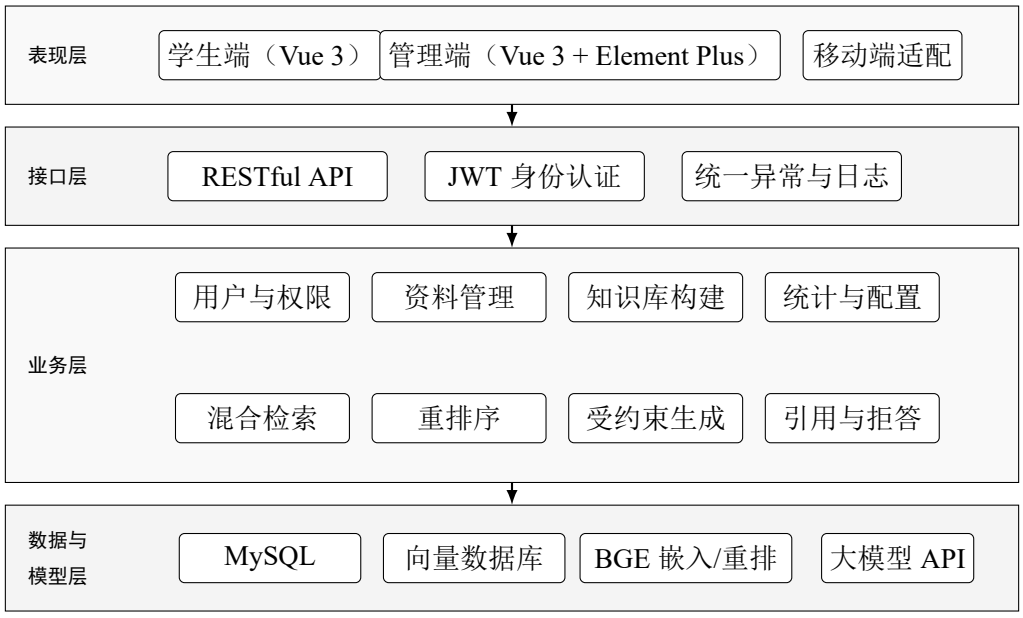


图 4-1 系统总体架构

表现层负责页面渲染和交互。学生端以对话页为主，包含问题输入、消息列表、引用卡片和历史会话；管理端采用侧边栏加内容区的布局，包含资料管理、知识库构建、用户角色、统计分析和系统配置。前端通过统一的请求封装处理令牌、错误码和加载状态，避免每个页面重复实现。

接口层负责协议转换、身份认证、参数校验和统一异常处理。所有接口返回统一结构，成功时包含数据，失败时包含错误码和提示信息；令牌解析、角色校验、请求编号生成和耗时统计都在这一层完成。接口层不写业务逻辑，只做“收请求、验身份、调服务、包响应”。

业务层是系统的核心，按领域划分为用户与权限、资料管理、知识库构建、混合检索、重排序、受约束生成、引用与拒答、统计与配置等模块。模块之间通过服务接口调用，不直接访问对方的数据库表。例如，问答模块只依赖“检索服务”的接口，不关心底层是 Chroma 还是其他向量库；生成模块只依赖“模型服务”的接口，不关心具体是大模型 API 还是本地模型。这样可以在不修改业务代码的情况下替换组件。

数据与模型层包括 MySQL、向量数据库、BGE 嵌入与重排模型、大模型 API。MySQL 保存用户、角色、文档、片段元数据、会话、消息、引用、反馈和配置；向量数据库保存片段向量和用于过滤的元数据；BGE 模型负责问题与片段的向量化以及候选重排；大模型 API 负责最终答案生成。模型服务通过超时、重试和熔断机制保护，避免单个外部服务故障拖垮整个问答链路。

在部署上，前端可以打包为静态资源，由 Nginx 托管；后端使用 Uvicorn 或 Gunicorn 运行多个 worker；MySQL 和向量数据库可以同机部署，也可以独立部署；模型服务通过内网或公网访问。对于本科课题的演示环境，单机部署即可满足要求；对于校内正式使用，可以把后端、数据库和模型服务拆分，通过负载均衡和连接池提高并发能力。

4.2 知识库与数据处理设计

知识库是问答系统的证据来源，其质量直接决定回答的上限。数据处理流程包括上传、格式校验、文本解析、清洗、结构分块、向量化和索引写入，如图 4-2 所示。

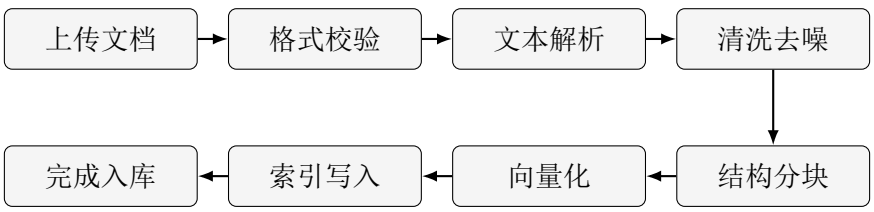


图 4-2 文档处理流程

格式校验首先检查扩展名和 MIME 类型，只允许 PDF、DOCX 和 TXT；然后检查文件大小和页数，避免超大文件拖垮解析服务。文本解析根据格式选择不同的库：PDF 使用 PyMuPDF 或 pdfplumber 提取文本和页码，DOCX 使用 python-docx 读取段落和表格，TXT 按编码探测后直接读取。解析结果保留页码、标题层级和段落顺序，为后续分块和引用定位提供依据。

清洗阶段去除页眉页脚、重复的空行、乱码和无关的广告信息，统一全角半角标点和空白字符。对于扫描版 PDF，文本提取可能为空，系统应提示用户该文件需要 OCR，暂不进入知识库；后续可以接入 OCR 服务作为扩展。清洗不能过度删除内容，例如条款编号、日期和金额往往是回答的关键，需要保留。

分块是数据处理中最影响效果的环节。固定长度切分实现简单，但容易把一条完整的规定或一个操作步骤拆到两个片段中；按段落切分可以保持语义完整，但段落长度差异大，过长的段落会超出嵌入模型输入上限。本课题采用结构感知的分块策略：先按标题层级把文档切成章节，再在章节内按段落和句子切分，目标长度为 500 至 800 个汉字，相邻片段保留 80 至 120 个汉字的重叠；对于表格和列表，尽量保持整表或整条不拆开；每个片段记录文档编号、标题路径、页码、分类和权限标签。设文档长度为 L，分块长度为 c，重叠长度为 o，则片段数量可按式 (4-1) 估算

$$n = \left\lceil \frac{L - o}{c - o} \right\rceil . \tag{4-1}$$

该式用于预估存储和向量化开销，实际数量还会受到标题边界和段落边界的影响。

向量化阶段调用 BGE 模型把每个片段编码为向量，写入向量数据库。为了支持增量更新，系统为文档和片段分配稳定的编号，重建时先按文档编号删除旧片段，再写入新片段；如果中途失败，任务状态标记为“失败”，保留旧索引，避免知识库出现一半新一半旧的中间状态。向量库中的每条记录包含片段编号、文档编号、向量、原文和元数据，元数据用于按分类、权限和更新时间过滤。

知识库构建采用异步任务。管理员上传文档后，接口立即返回文档编号和“待处理”状态，后台任务依次执行解析、分块、向量化和索引写入，前端轮询任务状态。任务表记录开始时间、结束时间、重试次数和失败原因，管理员可以查看失败详情并手动重试。对于批量上传，系统按文档逐个处理，单个文档失败不影响其他文档；对于重复上传，系统根据文件哈希判断是否已存在，若存在则提示用户选择覆盖或跳过。

4.3 检索增强问答流程设计

问答流程是系统的主链路，包括问题改写、双路召回、融合、重排、提示词组装、生成、引用校验和拒答。其时序关系如图 4-3 所示。

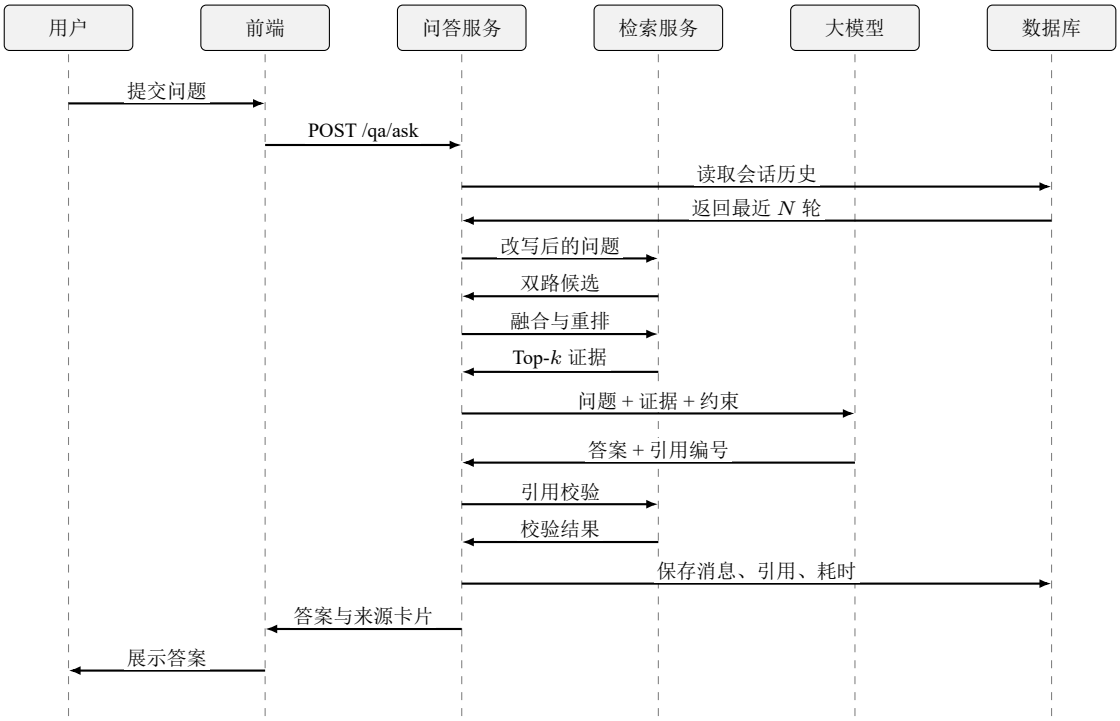


图 4-3 问答流程时序图

问题改写模块读取最近 N 轮会话，把当前问题、上一轮问题和系统回答一起送入改写模型，输出自包含的检索查询。改写只用于检索，最终回答仍以用户原始问题为准；如果改写结果与原问题差异过大，系统保留原始查询作为兜底，两路查询分别召回后合并，避免改写错误导致完全跑题。

双路召回包括 BM25 和向量检索。BM25 使用中文分词后的词项建立倒排索引，适合匹配文件编号、条款序号和专有名词；向量检索使用 BGE 模型编码问题和片段，适合处理同义表达和口语化提问。两路各取前 k_1 条候选，使用式 (2-3) 的 RRF 方法融合。融合后取前 k_2 条送入交叉编码器重排，重排分数低于阈值 τ 的候选被丢弃。若所有候选都被丢弃，系统判定证据不足，执行拒答。

提示词组装把系统角色、用户问题、候选证据和输出要求组合在一起。系统角色说明“你是校园问答助手，只能依据给定资料回答”；证据按编号排列，每条包含文档标题、章节路径和原文；输出要求包括“先回答结论，再说明依据”“每个关键结论标注引用编号”“不得编造资料中没有的信息”“证据不足时明确说

明”。为了降低模型忽略证据的风险，提示词把证据放在问题之前，并要求模型在答案末尾列出使用的引用编号，便于后端解析和校验。

生成后的引用校验分三步。第一步，检查答案中的引用编号是否都在候选证据编号集合内，越界的编号视为无效；第二步，检查每个被引用的片段是否仍然存在、当前用户是否有权限查看；第三步，检查答案中的关键结论是否能对应到至少一条引用片段，可以通过关键词覆盖或调用一次轻量判断模型完成。校验通过则返回答案和来源卡片；校验不通过则隐藏无效引用，并把答案降级为“以下是检索到的原文片段”，或者直接拒答。拒答不是失败，而是系统在证据不足时的保护机制，可以避免用户把无依据的内容当成学校规定。

检索与生成的主要参数如表 4-1 所示。这些参数不是固定不变的，需要根据知识库规模、文档长度和模型能力调整。例如，知识库较大时可以适当提高 k_1，但重排的 k_2 不宜过大，否则会增加时延并引入噪声；阈值 τ 过高会导致大量拒答，过低则会放过无关证据，需要通过测试集确定。

表 4-1 检索与生成的主要参数

参数	含义	建议取值
k_1	每个召回通道返回的候选数	20～30
k_2	进入重排的候选数	8～12
k	RRF 平滑常数	60
τ	重排相关性阈值	0.35～0.55
N	参与改写的最近会话轮数	3～5
T	大模型温度	0.1～0.3
$t_{\max}$	单次问答超时时间	8 s

4.4 数据库与接口设计

系统的核心实体包括用户、角色、文档、知识片段、会话、消息、引用和反馈。实体关系如图 4-4 所示。用户通过角色获得权限；用户拥有多个会话；会话包含多条消息；消息可以产生反馈；文档被切分为多个知识片段；消息通过引用关联到知识片段，引用表同时保存引用序号和原文快照，防止片段被删除后无法追溯。

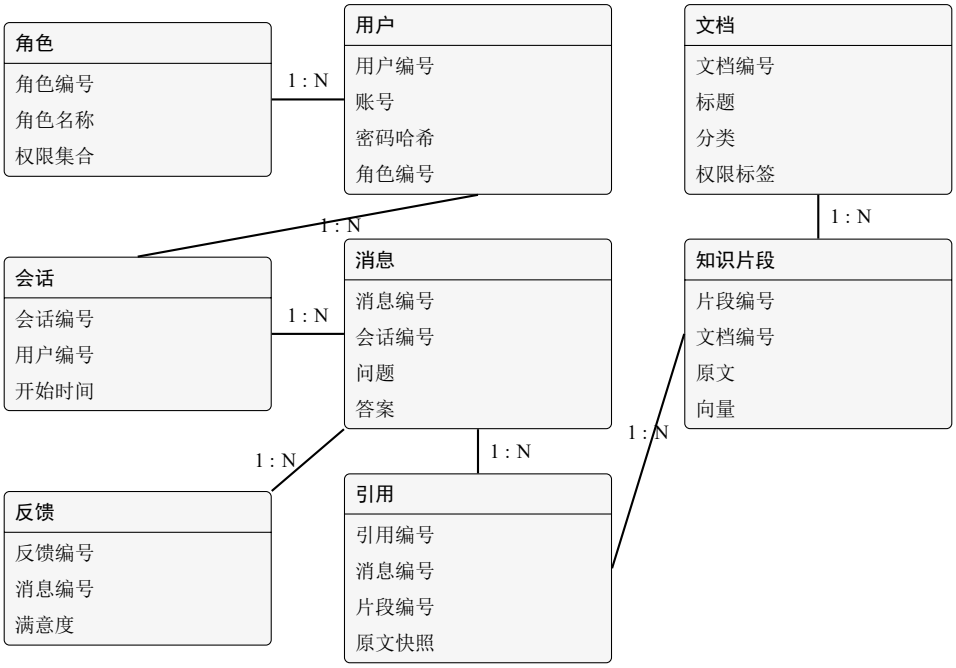


图 4-4 核心数据实体关系图

数据库表按领域划分。用户与权限域包含用户、角色、权限和角色权限四张表；资料域包含文档、片段和分类三张表；问答域包含会话、消息和引用三张表；反馈与配置域包含反馈、配置和任务三张表。核心表及主要字段如表 4-2 所示。表设计遵循第三范式，常用查询字段建立索引，例如文档的分类和状态、消息的会话编号和创建时间、引用的消息编号。

表 4-2 核心数据表设计

表名	主要字段	说明
`user`	id, username, password_hash, role_id, status, created_at	用户账号与角色
`document`	id, title, category_id, file_path, permission, status, hash	文档元数据与处理状态
`chunk`	id, document_id, content, page_no, heading_path, vector_id	知识片段及定位信息
`conversation`	id, user_id, title, created_at, updated_at	会话
`message`	id, conversation_id, role, content, latency, created_at	问题与答案
`citation`	id, message_id, chunk_id, quote, rank	答案引用的片段快照
`feedback`	id, message_id, score, comment, created_at	满意度反馈
`config`	key, value, description, updated_at	检索与模型参数
`task`	id, document_id, type, status, retry, error	异步任务状态

接口采用 RESTful 风格，按资源划分路径，使用 HTTP 方法表达操作语义。主要接口如表 4-3 所示。接口统一使用 JSON 请求和响应，分页参数为 `page` 和 `page_size`，错误响应包含 `code`、`message` 和 `request_id`。对于耗时操作，接口返回任务编号，由前端轮询任务状态，避免长时间阻塞 HTTP 连接。

表 4-3 主要 RESTful 接口

方法	路径	功能	权限
POST	/api/auth/login	登录并签发令牌	公开
GET	/api/documents	分页查询文档	管理员
POST	/api/documents	上传文档并创建解析任务	管理员
DELETE	/api/documents/{id}	删除文档及其片段	管理员
POST	/api/kb/rebuild	重建指定文档的索引	管理员
POST	/api/qa/ask	提问并返回答案与引用	登录用户
GET	/api/qa/history	查询当前用户的历史会话	登录用户
POST	/api/feedback	提交满意度反馈	登录用户
GET	/api/admin/stats	查询高频问题与使用统计	管理员
PUT	/api/admin/config	修改检索与模型参数	管理员

4.5 权限与安全设计

权限与安全设计贯穿认证、授权、数据访问和审计四个层面。认证解决“你是谁”，授权解决“你能做什么”，数据访问控制解决“你能看到哪些内容”，审计解决“你做过什么”。四者缺一不可。

认证采用 JWT。用户登录时，后端根据用户名查询用户记录，使用 bcrypt 校验密码；校验通过后，生成包含用户编号、角色编号、签发时间和过期时间的令牌，使用 HS256 或 RS256 签名；令牌返回前端后，前端在每次请求的 `Authorization` 头中携带 `Bearer` 令牌；后端依赖项解析令牌，校验签名、过期时间和用户状态，把当前用户注入请求上下文。令牌有效期设置为 2 小时，刷新令牌有效期为 7 天；用户修改密码或管理员禁用账号时，把用户编号写入失效列表，后端在校验时拒绝失效列表中的令牌。

授权采用 RBAC。系统预置学生、教师、管理员、超级管理员四个角色，权限以“资源: 操作”的形式定义，例如 `document:read`、`document:write`、`qa:ask`、`stats:read`。角色与权限多对多，用户与角色多对一。后端在路由上声明所需权限，依赖项在请求进入业务逻辑前完成校验；前端根据登录接口返回的权限集合控制菜单和按钮的显示。对于“只能操作自己的数据”这类要求，还需要在业务

层做数据级校验，例如学生只能查看自己的会话，教师只能查看授权范围内的统计。

数据访问控制重点在知识库检索。文档和片段都带权限标签，标签可以是“公开”“校内”“角色限定”。检索时，系统先把用户角色转换为可见标签集合，再在向量数据库的元数据过滤和 BM25 的查询条件中加入权限过滤，确保候选片段对当前用户可见。引用展示时再次校验片段权限，防止文档在会话期间被修改权限导致越权。对于管理员，可以查看所有公开和校内资料，但敏感资料仍需超级管理员授权。

其他安全措施包括：密码使用 bcrypt 加盐哈希，禁止明文存储和日志输出；数据库密码、JWT 密钥、大模型 API Key 通过环境变量注入，配置文件只保存占位符，仓库中不提交真实密钥；上传文件校验扩展名、MIME 类型、大小和页数，解析过程设置超时和内存上限；接口对问题长度、分页大小、检索条数做范围校验，防止资源耗尽；登录、上传、问答接口按用户和 IP 做频率限制；日志记录请求编号、用户编号、接口、耗时和错误码，对手机号、身份证号等敏感字段脱敏；定期备份 MySQL 和向量库，备份文件加密保存。上述措施不能完全消除风险，但可以覆盖本科课题演示环境的主要威胁。

第 5 章　系统实现

本章说明系统的开发环境、项目结构和关键模块的实现。重点不在逐行解释代码，而在说明每个模块如何落实第 4 章的设计：认证与授权如何拦截请求，文档如何被解析和分块，双路召回如何融合，提示词如何约束生成，引用如何校验，统计如何聚合。代码片段只保留与论文设计对应的核心部分，完整的配置、异常处理和前端页面不在正文中展开。

5.1 开发环境与项目结构

系统采用前后端分离结构，开发环境如表 5-1 所示。后端使用 Python 3.11，前端使用 Node.js 20，数据库为 MySQL 8.0，向量库为 Chroma，嵌入模型选用 BGE 中文小模型，重排模型选用对应的交叉编码器，大模型通过兼容 OpenAI 协议的 HTTP 接口调用。开发工具包括 VS Code、Navicat、Postman 和 Git。

表 5-1 开发环境

类别	配置
操作系统	Windows 11，开发与演示环境
后端	Python 3.11，FastAPI，Uvicorn，SQLAlchemy
前端	Node.js 20，Vue 3，Vite，Element Plus，Axios
数据库	MySQL 8.0，InnoDB，utf8mb4
向量库	Chroma，本地持久化模式
嵌入与重排	BGE 中文小模型，交叉编码器重排模型
大模型	兼容 OpenAI 协议的可配置 API
文档解析	PyMuPDF、python-docx、chardet
版本管理	Git，GitHub 远程仓库

项目分为后端、前端和部署三个部分。后端按领域划分目录：`api` 存放路由，

core 存放配置、安全和依赖，models 存放 SQLAlchemy 模型，schemas 存放请求与响应结构，services 存放认证、文档、检索、生成和统计服务，tasks 存放异步任务，utils 存放分块、文本清洗和引用解析工具。前端按页面和组件划分：views 存放登录、问答、历史、资料、统计和配置页面，components 存放消息气泡、引用卡片、上传组件和图表组件，api 封装接口请求，stores 管理用户和会话状态。部署部分包括 Nginx 配置、环境变量示例、数据库初始化脚本和启动说明。

配置通过环境变量注入，包括数据库连接、JWT 密钥、大模型地址与密钥、嵌入模型路径、向量库路径、检索参数和日志级别。仓库只提交.env.example，不提交真实密钥；本地开发使用.env，部署时由运行环境注入。这样做可以避免密钥进入版本历史，也便于在不同环境之间切换。

5.2 用户与权限管理

登录接口接收用户名和密码，先根据用户名查询用户记录，再使用 bcrypt 校验密码。校验通过后，服务读取用户的角色和权限，生成 JWT。令牌中只放用户编号、角色编号、签发时间和过期时间，不放密码、手机号等敏感信息。前端把令牌保存在内存或 sessionStorage 中，每次请求由 Axios 拦截器添加到 Authorization 头；令牌过期时，前端跳转登录页，避免在页面中反复弹出错误。

后端的认证与授权通过 FastAPI 依赖注入实现。认证依赖解析令牌、校验签名和有效期、查询用户状态；授权依赖检查当前用户是否具备路由声明的权限。核心代码如下所示。

```
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> User:
    payload = decode_token(token)
    user = db.get(User, int(payload["sub"]))
    if user is None or user.status != "active":
        raise HTTPException(status_code=401, detail="invalid token")
    return user


def require_permission(permission: str):
    def checker(user: User = Depends(get_current_user)) -> User:
        if permission not in user.permission_codes:
            raise HTTPException(status_code=403, detail="forbidden")
        return user
    return checker
```

路由上通过 Depends(require_permission("document:write")) 声明权限，

业务代码不需要重复判断角色。对于“只能查看自己的会话”这类数据级权限，服务层在查询时附加 user_id 条件，而不是先查出全部数据再在前端过滤。管理员接口还会记录操作日志，包括操作人、操作对象、操作类型和结果，便于审计。

前端根据登录接口返回的权限集合生成菜单。没有权限的菜单不渲染，按钮置灰或隐藏；但前端隐藏只是体验优化，真正的拦截在后端。令牌刷新采用双令牌机制：访问令牌过期后，前端用刷新令牌换取新的访问令牌；刷新令牌也过期时，跳转登录页。注销时，前端清除本地令牌，后端把当前令牌加入短期失效列表，避免令牌在有效期内被继续使用。

5.3 文档管理与知识库构建

文档上传接口接收 multipart 表单，字段包括文件、标题、分类、权限标签和备注。服务先计算文件 SHA-256，查询是否已有相同哈希的文档；若存在，返回重复提示，由用户选择覆盖或跳过。随后校验扩展名、MIME 类型、大小和页数，保存原文件到按日期分层的目录，并在 document 表插入记录，状态为“待处理”。接口立即返回文档编号，解析任务交给后台执行。

解析服务根据文件类型选择解析器。PDF 使用 PyMuPDF 逐页提取文本和页码，DOCX 使用 python-docx 读取段落、表格和样式，TXT 使用 chardet 探测编码后读取。解析结果统一为“块”列表，每个块包含文本、页码、样式层级和类型（标题、正文、表格、列表）。清洗时去除页眉页脚、连续空行和重复字符，保留条款编号、日期和金额等关键信息。

分块服务先按标题层级把块序列切成章节，再在章节内按段落和句子累积，直到达到目标长度。若当前段落加入后超过最大长度，则在句子边界处截断；相邻片段保留一定重叠，避免答案被切断。核心逻辑如下所示。

```
def structure_aware_chunks(blocks, max_len=800, overlap=100):
    chunks, buf, heading = [], [], ""
    for block in blocks:
        if block.kind == "heading":
            if buf: chunks.append(make_chunk(buf, heading)); buf = []
            heading = block.text.strip(); continue
        if len("".join(buf)) + len(block.text) <= max_len:
            buf.append(block.text)
        else:
            chunks.append(make_chunk(buf, heading))
            buf = ["".join(buf)[-overlap:], block.text]
    if buf: chunks.append(make_chunk(buf, heading))
    return chunks
```

每个片段写入 chunk 表，保存文档编号、原文、页码、标题路径和向量编号；

同时调用嵌入模型生成向量，写入 Chroma。写入采用“先删旧、再写新”的策略：重建某篇文档时，先按文档编号删除旧片段和旧向量，再批量写入新片段；如果中途失败，任务状态记为失败，旧索引保持不变，避免知识库出现不完整的中间状态。批量写入使用固定大小的批次，降低内存占用；嵌入模型不可用时，任务进入重试队列，最多重试三次，仍失败则通知管理员。

知识库构建页面展示文档列表、处理状态、片段数量、更新时间和操作按钮。管理员可以按分类、状态和关键词筛选，查看失败原因，单篇重试或批量重建。前端轮询任务状态，处理中显示进度条，成功后提示片段数量，失败时展开错误详情。对于扫描版 PDF，解析结果为空时提示“该文件可能需要 OCR”，并给出转为文本后重新上传的建议。

5.4 混合检索与重排序实现

检索服务对外提供统一的 `search(query, user, top_k)` 接口，内部依次执行 BM25 召回、向量召回、RRF 融合和交叉编码器重排。BM25 使用中文分词后的词项建立索引，索引在文档入库时增量更新；向量召回调用嵌入模型编码查询，再在 Chroma 中按余弦相似度取前 k_1 条。两路召回都带上权限过滤条件，确保候选片段对当前用户可见。

RRF 融合只依赖排名，不依赖原始分数，因此不需要对 BM25 分数和余弦相似度做归一化。实现时把两路召回结果转换为“片段编号列表”，再按式 (2-3) 累加分数。核心代码如下所示。

```
def reciprocal_rank_fusion(rankings, k=60):
    scores = defaultdict(float)
    for ranking in rankings:
        for rank, chunk_id in enumerate(ranking, start=1):
            scores[chunk_id] += 1.0 / (k + rank)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

融合后取前 k_2 条候选送入重排模型。重排模型把查询和片段拼接后编码，输出相关性分数；服务按分数降序排序，丢弃低于阈值 τ 的候选。若候选为空或全部低于阈值，返回空结果，由问答服务执行拒答。重排阶段设置超时，超时则退化为 RRF 排序，保证接口在模型服务不稳定时仍能返回结果。

为了观察检索效果，服务记录每次请求的查询、召回通道、候选数量、融合耗时、重排耗时和最终分数。日志按请求编号关联，便于分析“某类问题总是召回不到”或“重排把正确片段排到后面”的原因。管理端提供检索调试页面，管

理员输入问题后可以查看两路召回结果、融合排名和重排分数，辅助调整参数和补充资料。

5.5 受约束生成、引用溯源与多轮会话

生成服务把系统提示、证据、问题和输出约束组装成消息列表，调用大模型接口。系统提示明确角色和边界；证据按编号排列，每条包含标题、章节路径和原文；用户问题保留原始表述；输出约束要求先给结论、再给依据、关键结论标注引用编号、不得编造资料中没有的信息、证据不足时说明无法回答。提示词模板的核心部分如下所示。

```
SYSTEM_PROMPT = """You are a campus question-answering assistant.
Answer ONLY with the evidence below. Do not use outside knowledge.
Cite the evidence number in square brackets after each key claim.
If the evidence is insufficient, say you cannot answer and explain why.
"""

USER_PROMPT = """[Evidence]
{evidence}

[Question]
{question}

[Output]
1. A concise answer with citations like [1].
2. A short list of the evidence numbers you used.
"""
```

生成结果先经过引用解析。服务用正则表达式提取答案中的 [n] 编号，检查编号是否在候选证据集合内；再检查每个编号对应的片段是否存在、当前用户是否有权限查看；最后把答案中的关键句与引用片段做关键词覆盖检查。若引用编号越界或片段无效，服务隐藏无效引用，并把答案降级为摘录式回答；若答案没有任何引用，服务在末尾追加“以上内容未找到明确依据”，并标记为需人工检查。

多轮会话在生成前执行问题改写。服务读取最近 N 轮消息，把当前问题、上一轮问题和上一轮答案送入改写模型，要求输出一个不依赖上下文的自包含问题。改写结果用于检索，原始问题用于生成和日志。若改写结果为空或与原问题差异过大，服务保留原始问题作为兜底查询，两路查询分别召回后合并，避免改写错误导致完全跑题。会话消息保存问题时同时保存改写后的问题、引用片段和耗时，便于后续分析。

拒答策略根据证据情况分三级。第一级，检索为空或最高重排分数低于阈值，直接返回“暂时找不到相关依据”，并建议用户换一种问法或选择分类。第

二级，证据存在但引用校验失败，返回检索到的原文片段，并说明“以下为检索到的资料，未生成总结”。第三级，模型超时或返回格式错误，重试一次；仍失败则返回原文片段，并记录失败日志。三级策略的共同目标是：宁可少答，也不返回没有依据的内容。

5.6 管理统计与系统配置

统计服务按时间范围、分类和用户角色聚合问答数据，提供高频问题、未解决问题、知识库使用量、满意度和平均响应时延等指标。高频问题按规范化后的问题文本分组，去除标点和停用词，避免“补办学生证需要什么材料”和“补办学生证要哪些材料”被算成两个问题；未解决问题包括拒答、引用校验失败和用户点踩的消息；知识库使用量统计文档数量、片段数量、检索次数和命中片段分布；满意度按好评率和平均分展示；响应时延按检索、重排、生成三段拆分，便于定位瓶颈。

统计接口使用分页和聚合查询，避免一次性加载全部明细。高频问题按出现次数降序取前 20 条，未解决问题按时间倒序取前 50 条，趋势图按天聚合。管理端使用表格和图表展示，支持导出 CSV。对于数据量较大的场景，可以增加按天预聚合的统计表，定时任务把明细汇总为日指标，查询时直接读取汇总表。

系统配置页面提供检索条数、RRF 常数、重排阈值、模型名称、超时时间、拒答提示语等配置项。配置保存前做类型和范围校验，例如 k_1 必须在 5 到 100 之间，τ 必须在 0 到 1 之间，超时时间必须在 3 到 30 秒之间；保存后写入 `config` 表并刷新缓存，下一次请求生效。配置修改记录操作人和修改前后值，便于回滚。密钥类配置不在页面展示，只允许通过环境变量注入，避免管理员误操作或页面泄露。

第 6 章　系统测试与效果评价

测试是验证系统是否满足需求的必要环节。本章给出测试环境、功能与性能测试方案、问答测试集与评价指标、对照实验设计和结果分析方法。需要说明的是，本章的重点是建立可复现的测试方案和记录规范，所有指标均按系统实际运行结果计算；在系统联调完成前，不以估计值或主观判断代替实测数据。

6.1 测试环境与测试方案

测试环境分为服务端、客户端和数据三部分，如表 6-1 所示。服务端使用一台开发机运行后端、MySQL 和 Chroma，嵌入模型和重排模型在本地 CPU 上加载；客户端使用主流浏览器访问前端；测试数据包括校园公开制度、办事指南、常见问答和人工编写的测试问题。为了保证结果可复现，测试前固定模型版本、提示词版本、检索参数和知识库快照，测试过程中不修改配置。

表 6-1 测试环境

类别	配置
服务端	Windows 11，16 GB 内存，CPU 推理，Python 3.11
数据库	MySQL 8.0，Chroma 本地持久化
模型	BGE 中文嵌入模型、交叉编码器重排模型、可配置大模型 API
客户端	Chrome 120 及以上，1920×1080 分辨率
网络	校园网或家用宽带，记录上下行带宽和接口时延
知识库	校园公开制度、办事指南、常见问答，按教务、学工、后勤、图书馆分类
测试集	100 道校园问题，四个主题各 25 道，人工标注标准答案和来源

测试分为四类。第一，功能测试。按照需求清单逐项验证登录、上传、解析、

问答、引用、历史、反馈、统计和配置等功能，检查正常流程和异常流程是否返回预期结果。第二，性能测试。使用脚本模拟并发请求，记录平均响应时间、95 分位响应时间、错误率和资源占用；问答接口的时延拆分为检索、重排、生成三段，分别统计。第三，效果测试。在标注好的测试集上运行不同方案，计算检索命中率、答案正确率、引用正确率和拒答准确率。第四，安全测试。检查未登录访问、越权访问、令牌过期、恶意文件上传、超长输入和频繁请求等场景，确认系统返回正确的错误码且不泄露敏感信息。

功能测试采用“用例—操作—预期结果—实际结果”的记录方式。表 6-2 给出主要测试用例的设计，实际执行时逐条填写通过与否、实际输出、耗时和截图编号。对于失败用例，记录复现步骤、日志请求编号和初步原因，修复后回归测试。异常场景与正常场景同等重要，例如检索为空时是否拒答、模型超时是否降级、引用越界是否隐藏，这些行为直接关系到系统是否可信。

表 6-2 主要功能测试用例设计

编号	测试项	操作或输入	预期结果
T-01	正常登录	输入正确账号和密码	返回令牌，进入问答页
T-02	密码错误	输入错误密码	提示账号或密码错误，不签发令牌
T-03	越权访问	学生令牌调用资料删除接口	返回 403，不执行删除
T-04	上传 PDF	上传一份可解析的 PDF	返回文档编号，状态转为成功
T-05	上传非法文件	上传可执行文件或超大文件	拒绝上传，提示格式或大小错误
T-06	解析失败	上传损坏的 DOCX	状态为失败，展示原因并可重试
T-07	正常问答	提问“补办学生证需要什么材料”	返回答案、引用编号和来源卡片
T-08	证据不足	提问知识库中没有的内容	拒答，提示换一种问法
T-09	引用校验	构造模型返回越界引用	隐藏无效引用，降级为摘录回答
T-10	多轮追问	先问助学金条件，再问“需要哪些材料”	改写后检索到助学金材料，答案连贯
T-11	历史查询	打开历史页并按关键词筛选	只显示当前用户的会话
T-12	统计与配置	管理员查看高频问题并修改 k_1	图表更新，配置保存并校验范围

性能测试使用 Locust 或 JMeter 编写脚本，分别设置 1、5、10、20 个并发用

户，持续运行 5 分钟，记录请求成功率、平均时延、95 分位时延和吞吐量。由于大模型调用受外部服务影响，性能测试分为“仅检索”和“完整问答”两组：仅检索组不调用大模型，用于评估本地检索链路的性能；完整问答组调用大模型，用于评估端到端体验。两组都记录失败请求的错误码和超时比例。测试期间监控 CPU、内存和网络占用，避免把资源不足导致的超时误判为代码问题。

6.2 问答测试集与评价指标

测试集从校园常见问题中整理，覆盖教务管理、校园生活、实验室使用和办事流程四个主题，每个主题 25 道，共 100 道。题目来源包括任务书和开题报告中的场景、学校公开的办事指南、学生访谈中提出的问题，以及作者在日常学习中遇到的实际问题。每道题记录问题、标准答案、来源文档、答案要点和标注依据；对于有多种合理表述的题目，标准答案给出要点集合，而不是唯一字符串。测试集构成如表 6-3 所示。

表 6-3 问答测试集构成

主题	题量	示例
教务管理	25	补办学生证、成绩单打印、转专业申请、考试违纪处理
校园生活	25	宿舍报修、校园网连接、电费充值、心理咨询预约
实验室使用	25	实验室预约、设备借用、安全考试、开放时间
办事流程	25	在读证明、党员材料、档案转递、离校手续

评价指标包括检索命中率、答案正确率、引用正确率、拒答准确率和平均响应时延。检索命中率衡量前 k 条候选是否包含标注来源片段，定义为

$$\text{HitRate@}k = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left(\mathcal{C}_i(k)\cap\mathcal{G}_i\neq\varnothing\right), \tag{6-1}$$

其中 N 为题目数量，$\mathcal{C}_i(k)$ 为第 i 题的前 k 条候选片段集合，$\mathcal{G}_i$ 为该题的标准来源片段集合，$\mathbb{I}(\cdot)$ 为指示函数。答案正确率按人工判定，答案覆盖标准答案的全部要点记为正确，覆盖部分要点记为部分正确：

$$\text{Acc} = \frac{N_{\text{correct}} + 0.5N_{\text{partial}}}{N}. \tag{6-2}$$

引用正确率衡量答案中的引用是否真实支持对应结论，定义为

$$\mathrm{CiteAcc} = \frac{N_{\text{valid citation}}}{N_{\text{total citation}}}, \tag{6-3}$$

其中有效引用指引用编号存在、片段可访问、且片段内容与所支持结论一致。拒答准确率分别统计“无答案题目正确拒答”和“有答案题目未误拒”的比例，用两者的平均值表示：

$$\mathrm{RefusalAcc} = \frac{1}{2}\left(\frac{N_{\text{refuse correct}}}{N_{\text{unanswerable}}} + \frac{N_{\text{answer correct}}}{N_{\text{answerable}}}\right). \tag{6-4}$$

平均响应时延按检索、重排、生成三段分别统计，并计算端到端平均值和 95 分位值。时延只统计成功返回的请求，超时和错误请求单独统计比例。

人工标注采用双人独立标注、分歧讨论的方式。标注前统一答案要点和引用有效的判定标准；标注时只看问题、答案和引用原文，不看系统名称和实验分组，减少主观偏差。对于争议样本，由第三人复核并记录理由。测试集和标注结果保存在版本库中，实验后不再修改，保证不同方案在同一数据上比较。

6.3 对照实验设计

对照实验围绕两个问题展开：检索增强是否比纯大模型回答更准确、更可核查；不同的检索策略对效果有什么影响。实验设置五个组，如表 6-4 所示。A 组为纯大模型直答，不提供任何校园资料；B 组为 RAG 问答，按检索策略分为 B1（仅向量检索）、B2（BM25 与向量混合）、B3（混合检索加重排序）。A 组用于观察无外部知识时的表现，B 组内部用于比较检索策略的贡献。

表 6-4 对照实验分组

组别	检索与生成方式	观察目的
A	纯大模型直答，不提供证据	无校本知识时的准确性与引用能力
B1	仅向量召回，Top-k 后直接生成	稠密检索的基线效果
B2	BM25 与向量双路召回，RRF 融合	混合检索是否提高召回质量
B3	混合检索后交叉编码器重排	重排序是否提高顶部证据相关性

实验控制以下变量：使用同一套测试题、同一版本的大模型、同一温度和最大生成长度、同一套输出格式要求；B 组使用同一知识库快照，只是检索策略不同；每组运行一次完整测试集，记录原始输出、引用、耗时和错误。为了减少顺序效应，测试题按随机顺序执行；对于可能受缓存影响的指标，实验前清空检索缓存。A 组不提供证据，因此不统计引用正确率，只统计答案正确率和响应时延；B 组统计全部指标。

实验流程分为四步。第一步，构建并冻结知识库，记录文档数量、片段数量和分类分布。第二步，运行 A 组和 B 组，保存每道题的答案、引用、候选片段和耗时。第三步，人工标注答案正确性和引用有效性，标注时不看组别。第四步，汇总指标，按主题和题型分组分析，找出失败案例的共同原因。对于差异明显的指标，进一步做参数敏感性实验，例如改变 k_1、k_2、τ 和分块长度，观察指标变化。

6.4 结果记录与分析方法

实验结果按“原始记录—汇总指标—失败分析”三层组织。原始记录包括每道题的组别、答案、引用、候选片段、耗时和错误码；汇总指标按组别和主题计算平均值、标准差和样本数；失败分析把错误分为检索未命中、证据不完整、生成偏离证据、引用错误、拒答误判五类，统计各类占比并给出典型案例。这样既能看出整体差异，也能定位问题出在检索还是生成。

分析时需要注意三点。第一，区分“答案正确”和“引用正确”。一个答案可能内容正确但引用错误，也可能引用正确但答案不完整，两者要分开统计。第二，关注拒答的两类错误。把可回答的问题误拒，会降低可用性；把不可回答的问题强行回答，会带来错误信息风险，两类错误的代价不同，需要结合使用场景权衡。第三，避免只报告平均值。不同主题的题目难度差异较大，按主题分组报告可以避免某一类题目拉高或拉低整体指标。

受毕业设计进度限制，完整的对照实验将在系统联调完成后执行。本节给出的测试集、指标、分组和记录方法已经固定，实验执行时只需按表中字段填写实际结果，不需要重新设计评价方案。所有数据以系统实际输出和人工标注为准，不以预期值、经验值或他人系统的结果替代；对于无法复现或超出解释范围的结果，在论文中如实说明。

结　论

本文围绕校园信息分散、重复咨询多、通用大模型回答缺乏校本依据等问题，设计并实现了一个基于大模型的校园智能问答系统。系统以学校公开的规章制度、办事指南和常见问答为知识源，把“资料入库—问题理解—混合召回—融合重排—受约束生成—引用返回”串成一条完整链路，力求让回答既有自然语言的易读性，又能追溯到具体资料。

在需求分析阶段，本文从技术、经济、操作和进度四个方面论证了可行性，划分了学生、教师、管理员和超级管理员四类角色，梳理了登录与权限、资料上传与分类、解析与知识库构建、自然语言问答、引用溯源、历史与反馈、统计与配置等功能需求，并提出了响应时间、并发、可靠性、安全性、可维护性和可扩展性等非功能要求。在系统设计阶段，本文采用前后端分离的分层架构，把表现层、接口层、业务层、数据与模型层分离，设计了结构感知分块、BM25与向量双路召回、RRF 融合、交叉编码器重排、提示词约束、引用校验和分级拒答等关键流程，并给出了数据库表、RESTful 接口和权限安全方案。在实现阶段，本文说明了 FastAPI 认证依赖、文档解析与分块、RRF 融合、提示词组装和引用校验等核心代码结构，以及管理统计和配置的实现方式。

本文的主要工作可以概括为三点。第一，针对校园文档的标题层级和条款结构，设计了保留标题路径、页码和权限标签的分块方法，使检索片段既能保持语义完整，又能定位到原文。第二，把稀疏检索和稠密检索结合起来，先用 RRF 融合候选，再用交叉编码器重排，在不显著增加成本的前提下提高顶部证据的相关性。第三，把引用溯源和拒答作为生成流程的组成部分，通过提示词约束、引用编号校验和分级降级，降低模型编造内容的概率，让用户能够核对答案来源。

本文还存在一些不足。首先，系统核心模块的实现方案和测试方案已经形成，但完整的联调、性能测试和对照实验仍在推进，实测数据需要在后续工作中补充；本文没有用估计值代替实验结果。其次，知识库目前以文本类文档为主，对扫描版 PDF、图片表格和富媒体内容的支持不够，后续可以接入 OCR 和多模态模型。再次，嵌入模型和重排模型在 CPU 上运行，面对大规模知识库时检索

时延会上升，需要通过模型量化、缓存或 GPU 推理优化。最后，多轮对话的问题改写仍依赖大模型，改写错误可能影响召回，后续可以结合规则和用户反馈做更稳健的处理。

后续工作可以从四个方向展开：一是扩充校园知识库，建立资料更新和责任分工机制，保证知识库与学校最新制度同步；二是完善测试集和标注规范，扩大样本规模，增加跨主题、对比式和否定式问题，提高评价的覆盖面；三是研究自适应检索，根据问题类型动态调整召回通道、候选数量和重排阈值，减少不必要的模型调用；四是把系统部署到校园内网，结合实际使用数据观察用户行为，持续改进回答质量和引用展示。通过上述工作，系统有望从毕业设计原型逐步发展为可实际使用的校园信息服务平台。

参考文献

[1] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33. Red Hook, NY: Curran Associates, 2020: 9459-9474.

[2] 白晶, 张艳明, 扈韵绮, 等. 基于大语言模型和 RAG 技术的高校知识库智能问答系统构建与评价 [J]. 科技视界, 2026(3): 110-114.

[3] 袁乐, 刘绍华, 王禹, 等. 大语言模型检索增强生成优化技术研究综述 [J]. 计算机学报, 2026, 49(2): 383-422. DOI:10.11897/SP.J.1016.2026.00383.

[4] 张犬俊, 谢杨, 房春荣, 等. 检索增强生成在软件工程中的应用综述 [J]. 软件学报, 2026, 37(3): 1316-1339. DOI:10.13328/j.cnki.jos.007567.

[5] ASAI A, WU Z, WANG Y, et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection[C]//The Twelfth International Conference on Learning Representations. 2024.

[6] KARPUKHIN V, OGUZ B, MIN S, et al. Dense Passage Retrieval for Open-Domain Question Answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Stroudsburg, PA: Association for Computational Linguistics, 2020: 6769-6781. DOI:10.18653/v1/2020.emnlp-main.550.

[7] XIAO S, LIU Z, ZHANG P, et al. C-Pack: Packed Resources for General Chinese Embeddings[C]//Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2024: 641-649. DOI:10.1145/3626772.3657878.

[8] 郜洪奎, 马瑞祥, 包骐豪, 等. 基于混合检索增强的双塔模型研究 [J]. 计算机科学, 2025, 52(6): 324-329. DOI:10.11896/jsjkx.240800017.

[9] 张健, 唐晋韬, 王挺, 等. 基于混合检索重排序策略的大模型增强方法 [J]. 中文信息学报, 2025, 39(4): 42-54.

[10] 孙飒, 张经伟, 江卫. 面向企业知识库的层次化分块与混合检索增强生

成方法设计 [J]. 通信技术, 2026, 59(7): 745-754. DOI:10.3969/j.issn.1002-0802.2026.07.002.

[11] 宋保仁, 冷静. 检索增强生成技术在图书馆私有知识库构建中的应用与边界——以四川高校特色数据库为例 [J]. 情报理论与实践, 2026, 49(4): 184-193. DOI:10.16353/j.cnki.1000-7490.2026.04.021.

[12] 秦晓瑞, 何柳, 安然, 等. 面向航空标准的大语言模型迭代检索增强生成方法 [J]. 南京航空航天大学学报（自然科学版）, 2026, 58(1): 235-248. DOI:10.16356/j.2097-6771.2026.01.022.

[13] 甘克勤, 高亮, 肖宝坤, 等. 基于检索增强生成与对话历史管理的标准智能问答系统研究 [J]. 中国标准化, 2026(2): 43-47. DOI:10.3969/j.issn.1002-5944.2026.02.002.

[14] 罗云, 徐嘉骏, 彭明阳, 等. 忠实性增强的国防政策检索增强生成方法 [J]. 火箭军工程大学学报, 2026, 40(1): 132-142. DOI:10.20189/j.cnki.CN/61-1527/E.202601012.

[15] 董永涛, 崔翛龙. 基于大模型和检索增强生成技术的舰艇装备故障智能问答系统 [J]. 中国舰船研究, 2025, 20(6): 144-153. DOI:10.19693/j.issn.1673-3185.04285.

[16] 彭琳. 基于检索增强生成的企业级知识库系统构建与优化 [J]. 电脑知识与技术, 2026, 22(12): 35-37.

[17] 陶金涛, 张楠楠, 廖世斌. 基于检索增强生成与大语言模型的新疆地质矿产智能问答系统构建研究 [J]. 新疆地质, 2026, 44(1): 154-159.

[18] 王祥昊, 王莉, 胡恒闯, 等. 基于知识检索增强的铁路应急处置策略智能生成方法 [J]. 中国铁道科学, 2026, 47(1): 233-243. DOI:10.3969/j.issn.1001-4632.2026.01.21.

[19] 窦嘉铭, 王小君, 乔骥, 等. 基于知识图谱检索增强生成与大语言模型的配网检修智能辅助决策框架 [J]. 工程科学与技术, 2026, 58(4): 67-79.

[20] ES S, JAMES J, ESPINOSA-ANKE L, et al. RAGAs: Automated Evaluation of Retrieval Augmented Generation[C]//Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. Stroudsburg, PA: Association for Computational Linguistics, 2024: 150-158. DOI:10.18653/v1/2024.eacl-demo.16.

[21] 卢万有. 基于 JWT 的 RBAC 在前后端分离项目中的设计与实现 [J]. 电脑编程技巧与维护, 2025(1): 46-48.

[22] 廖明焱. 面向企业应用的 RAG 知识安全检索与 AI 脱敏技术研究 [J]. 电脑编程技巧与维护, 2026(8): 126-128.

[23] 魏巍. 基于 RBAC 模型云资源管理系统访问控制权限的设计与实施 [J]. 科学技术创新, 2025(2): 122-125.

[24] 张宁. 基于 LangChain 的 Python 知识问答系统的设计与实现 [J]. 现代信息科技, 2026, 10(5): 66-71. DOI:10.19850/j.cnki.2096-4706.2026.05.013.

[25] 吴海周, 黄俊, 张佳晨. 基于 RAG 机制的知识库问答性能优化与参数调控研究 [J]. 工业控制计算机, 2026, 39(7): 111-113.

[26] LangChain. LangChain Documentation[EB/OL]. [2026-09-21]. https://python.langchain.com/docs/.

致　谢

本论文的完成离不开许多人的帮助。首先感谢我的指导教师段雪丽老师。从选题、开题到系统设计和论文撰写，段老师多次提出具体而中肯的意见，帮助我厘清研究问题、收缩功能范围，并在论文结构、引用规范和实验设计上给予细致指导。每当我陷入“功能做得越多越好”的思路时，段老师都会提醒我先保证核心链路完整、证据可追溯，这让我少走了不少弯路。

感谢信息工程学院的各位老师。在本科阶段的学习中，软件工程、数据库、Web开发、数据结构与算法等课程为本文的系统设计和实现打下了基础；开题答辩组的老师对研究范围、技术路线和评价方法提出了宝贵建议，使我对课题的难点有了更清楚的认识。

感谢在需求和测试环节提供帮助的同学和朋友。他们从学生的角度提出了许多真实的校园问题，帮助我梳理问答场景和测试题目；在系统试用过程中，他们反馈了界面、响应速度和引用展示方面的问题，这些反馈对改进设计很有价值。

感谢我的家人一直以来的支持和鼓励。在毕业设计期间，他们给了我稳定的生活环境和充分的理解，让我能够安心完成学业。

最后，感谢各位评审老师在百忙之中审阅本文。由于本人的知识和经验有限，论文和系统中难免存在不足，恳请各位老师批评指正。